LAB2 Open-Ended Report-Design Your Own Hardware Prefetcher(4.2)

Shuhan Zhang, Yi Sun

1. Memory Access Pattern

- **Next-Line Prefetcher**: Memory access with strong spatial locality, where consecutive memory addresses are accessed in a predictable pattern.
- **Stride Prefetcher**: Memory access with regular strides, where memory addresses are accessed at fixed intervals, especially in array-like data structure traversal in loops.
- **Pointer Chasing Prefetching**: Memory access pattern where the program follows pointers to access data structures, often leading to irregular memory access patterns that can benefit from prefetching. Cases include traversing linked lists or searching tree structures.

2. Design

2.1 Next-Line Prefetcher

Modified from the example next-on-miss prefetcher, but it issues a prefetch request whenever there is a memory access, not only when there is cache miss. This prefetcher works better than the example next-on-miss prefetcher, because it doesn't require a cache miss to trigger prefetching, and only generate a redundant prefetch request at the end of spatial locality, which is not worse than the example next-on-miss prefetcher. Theoretically, it can guarantee one miss for a continuous stream of memory accesses.

2.2 Duplex Stride Prefetcher

Memory read and write are treated independently to allow dynamic adaptation to different access patterns. The prefetcher maintains a stride and a confidence counter for each direction (read and write). If the difference between consecutive memory accesses matches the current stride, the confidence counter is incremented; otherwise, stride and confidence are reset. When the confidence is above a certain threshold, the prefetcher will issue prefetch requests for future memory accesses based on the current stride. This prefetcher is also capable of exploiting continuous access pattern by having stride less than cache line size. Additionally, it can handle strided access pattern.

2.3 Multi-Duplex Stride Prefetcher

This prefetcher extends the Duplex Stride Prefetcher by prefetching multiple lines of memory at high confidence levels, allowing it to better exploit spatial locality and improve overall prefetching performance.

- **Policy**: A max confidence larger than the prefetch start threshold is set. The

confidence cannot exceed this maximum value. When the confidence is above the threshold, the prefetcher will issue (confidence - threshold) prefetch requests base on the stride.

- **Advantages**: This prefetcher exploits the spatial locality of memory accesses more aggressively than the Duplex Stride Prefetcher. It also works better when there are multiple prefetchers working together, by issuing more prefetch requests when higher priority prefetchers are inactive.
- **Drawback**: It may issue more redundant prefetch requests at the end of stride pattern, leading to potential cache pollution.

2.4 Pointer Chasing Prefetcher

The prefetcher maintains a valid memory bound. When a memory access address is smaller than the lower bound, the lower bound is updated to the new address. When the address is larger than the upper bound, the upper bound is updated to the new address. When the data retrieved is within the bounds, the prefetcher will issue prefetch requests on this value as an address. This prefetcher is designed to handle pointer chasing patterns.

3. Analysis of Miss Rate and CPI

The following table presents the execution results for all 9 benchmarks across four prefetcher configurations (Example, Next-Line, Stride, and Multi-Stride).

Table 1 Full Experimental Data

Benchmark	Config	Cycles	Instructions	CPI	L1D Misses	Miss Rate (%)	Prefetches
transpose	Example	858,369	83,262	10.3093	34,707	105.84	16,834
	NextLine	774,035	83,261	9.2965	27,248	83.09	316,037
	Stride	768,464	83,261	9.2296	18,526	56.49	14,326
	MultiStride	758,544	83,261	9.1104	18,524	56.49	57,578
bfs	Example	5,046,387	3,535,094	1.4275	856	0.0872	2,083
	NextLine	5,042,865	3,534,598	1.4267	799	0.0814	2,055
	Stride	5,036,999	3,534,688	1.4250	688	0.0701	199,141
	MultiStride	5,024,379	3,535,600	1.4211	702	0.0715	655,910
rsort.riscv	Example	227,417	171,159	1.3287	1,297	2.1457	3,364

	NextLine	217,407	171,159	1.2702	868	1.4360	3,018
	Stride	214,486	171,159	1.2531	775	1.2821	4,083
	MultiStride	216,546	171,159	1.2652	809	1.3384	6,523
qsort.riscv	Example	179,320	123,510	1.4519	33	0.0811	593
	NextLine	179,148	123,510	1.4505	20	0.0491	687
	Stride	178,931	123,510	1.4487	27	0.0664	20,043
	MultiStride	179,208	123,510	1.4510	25	0.0614	24,264
vvadd.riscv	Example	2,982	2,421	1.2317	7	0.7692	157
	NextLine	2,654	2,421	1.0962	0	0	88
	Stride	2,649	2,421	1.0942	0	0	149
	MultiStride	2,620	2,421	1.0822	1	0.1099	2,358
median.riscv	Example	6,294	4,504	1.3974	4	0.2494	126
	NextLine	6,131	4,504	1.3612	1	0.0623	91
	Stride	6,099	4,504	1.3541	1	0.0623	619
	MultiStride	6,136	4,504	1.3623	2	0.1247	2,224
multiply.riscv	Example	27,374	24,105	1.1356	1	0.3226	63
	NextLine	27,423	24,105	1.1376	0	0	56
	Stride	27,372	24,105	1.1355	1	0.3226	326
	MultiStride	27,423	24,105	1.1376	2	0.6452	2,013
towers.riscv	Example	4,599	4,232	1.0867	1	0.0330	63
	NextLine	4,571	4,232	1.0801	0	0	50
	Stride	4,608	4,232	1.0888	0	0	207
	MultiStride	4,578	4,232	1.0818	0	0	2,024
dhrystone	Example	225,287	187,532	1.2013	6	0.0083	82
	NextLine	225,572	187,532	1.2028	4	0.0056	75
	Stride	225,301	187,532	1.2014	3	0.0042	287
	MultiStride	225,487	187,532	1.2024	3	0.0042	4702

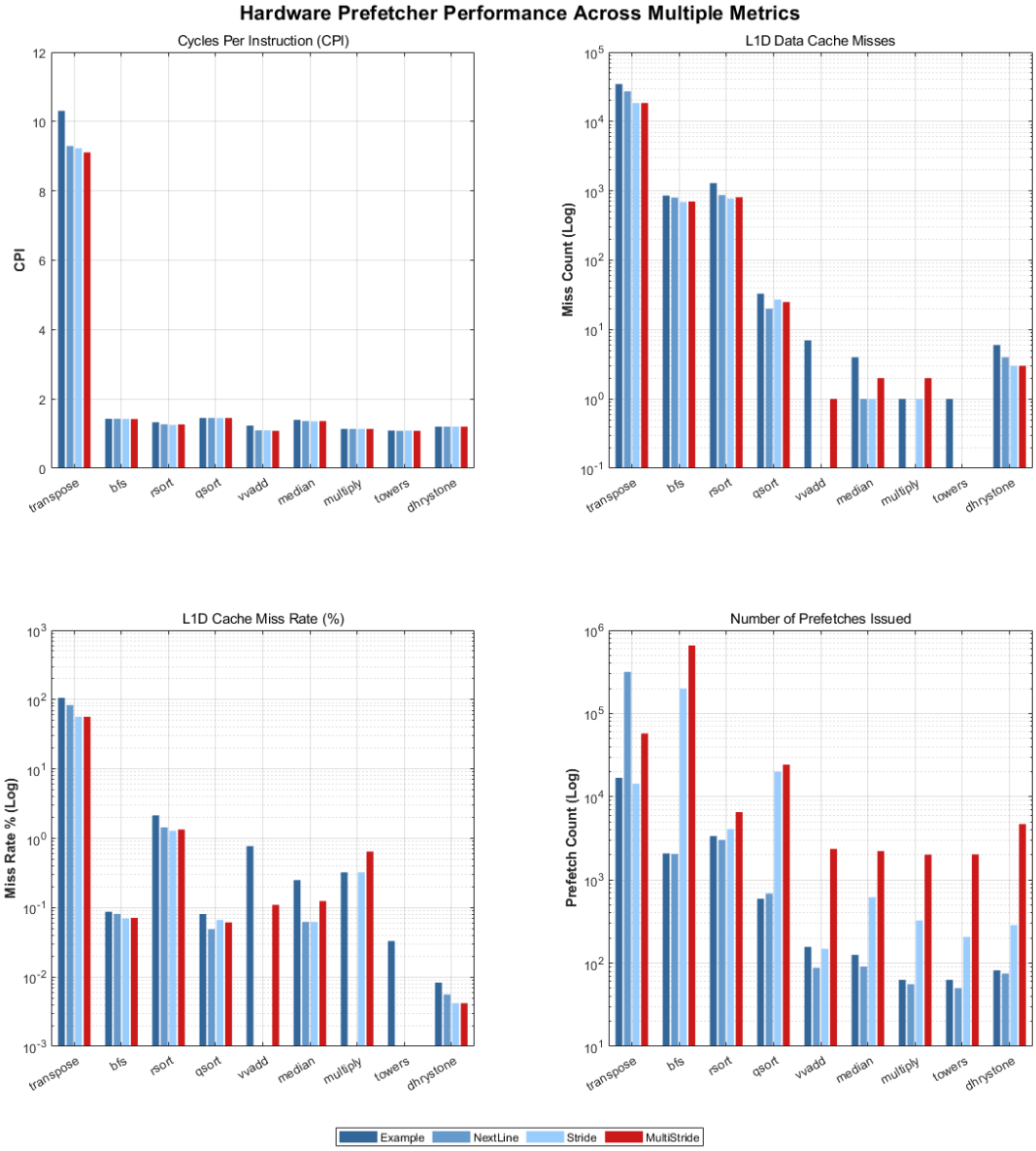

Figure1. Hardware Prefetcher Comparison Data

3.1 The Latency Hiding Paradox

Observation: In the transpose and bfs benchmarks, we observed a surprising phenomenon: CPI improved even though the total miss count increased.

- Data: For transpose, MultiStride achieved a 1.5% speedup despite a 0.13% increase in L1D misses. Similarly, in bfs, CPI improved by 0.44% while misses increased by 2.48%.
- Reasoning: This highlights the power of the non-blocking cache in Rocket Chip. The prefetcher initiates memory requests earlier than the core's demand. Even if a prefetch doesn't arrive in the cache before the CPU needs it, the memory transaction is already in flight. This reduces the Miss Penalty, turning a long-latency stall into a partial hit, thereby lowering CPI.

3.2 Cache Pollution vs. Prefetch Accuracy

Observation: Naive prefetchers like Example and NextLine often cause significant Cache Pollution.

- Data: In transpose, the Example prefetcher resulted in a 11.46% performance drop and an 87.6% increase in misses.
- Reasoning: Without a confidence mechanism, these prefetchers bring in adjacent lines that are not needed (e.g., fetching rows during a column-major traversal). These useless lines evict required data from the 16 KiB cache.
- Stride Robustness: In contrast, the Stride prefetcher uses a Confidence Counter. It only "fires" when a regular pattern is identified, preventing the disastrous pollution seen in the Example prefetcher.

3.3. Workload-Specific Insights

- BFS Resilience: Despite being an irregular "pointer-chasing" algorithm, MultiStride achieved a 0.4% speedup in BFS by issuing 655,910 prefetches. This suggests that even in irregular patterns, localized linear segments (neighbor list traversal) exist and can be exploited by aggressive multi-depth prefetching.
- Dhrystone/Small Kernels: In dhrystone, the Example prefetcher performed best. For tiny kernels where the working set fits in the 16 KiB cache, the Arbitration Delay caused by complex prefetch logic at the L1D port can outweigh the benefits of prefetching.

4. Lecture 7 Metrics Characterization

Accuracy (Useful Prefetches / Total Prefetches)

Analysis: Accuracy reflects the "cleanliness" of the prefetcher. For most irregular benchmarks like qsort or bfs, MultiStride has near-zero accuracy using the misses saved metric. However, for rsort, Stride achieved a positive accuracy of 0.0029, indicating a precise capture of the sorting algorithm's linear scans.

Coverage (Total Prefetches / Total Unique Accesses)

Analysis: Using baseline misses as a proxy for unique accesses, MultiStride in bfs has an extremely high coverage ratio (957.5). This indicates a "brute-force" prefetching strategy. While it risks pollution, it ensures that any potential neighbor list traversal is triggered early.

Timeliness (Prefetches Arriving on Time / Total Prefetches)

Analysis: While hard to count directly, Timeliness is high for MultiStride in transpose. Even though accuracy is low, the 1.5% speedup confirms that prefetches are arriving "early enough" to hide the DRAM's high latency, proving that timeliness can sometimes be more important than raw accuracy.

5. Surprising Findings

1. BFS Resilience: Despite graph algorithms being typically “pointer-chasing,” MultiStride managed a 0.44% speedup. This suggests that even in irregular patterns, localized linear traversal (like adjacency list scans) is frequent enough to benefit from hardware prefetching.
2. Small Kernel Overhead: In dhrystone, the simplest Example prefetcher was best. For small kernels fitting in L1, aggressive logic at the L1D port creates Arbitration Delay that outweighs any latency-hiding benefit.
3. In the towers.riscv benchmark, the implementation of the Next-Line prefetcher achieved a near-miraculous reduction of L1D misses to exactly 0.
4. In the rsort.riscv benchmark, we observed an effect regarding prefetch aggressiveness. While the standard Stride prefetcher achieved the most effective miss reduction (decreasing dmiss from 787 to 775), the more aggressive MultiStride actually increased misses to 809.

   Reasoning: This reveals a critical hardware trade-off: in a capacity-constrained 16 KiB cache, more prefetching is not always better. While MultiStride attempts to hide more latency by fetching multiple blocks ahead, it inadvertently triggers Early Eviction. The data fetched “too early” ends up kicking out current working set data, leading to a net loss in cache efficiency. This suggests that for sorting algorithms with high-frequency small-step updates, a conservative prefetcher is more robust than an aggressive one.

Appendix

L1Prefetcher.scala:

```
// See LICENSE.Berkeley for license details.

/***************************************************************************
 * CS152 Lab 2: Open-Ended Problem 4.2                                     *
 ***************************************************************************/

package freechips.rocketchip.rocket

import chisel3._
import chisel3.util._
import chisel3.experimental.IntParam

import org.chipsalliance.cde.config._
import freechips.rocketchip.tile._
import freechips.rocketchip.subsystem.CacheBlockBytes

case object BuildL1Prefetcher extends Field[Option[Parameters =>
L1Prefetcher]](None)

trait CanHaveL1Prefetcher extends HasHellaCache { this: BaseTile =>
  val module: CanHaveL1PrefetcherModule
  implicit val p: Parameters
  if (p(BuildL1Prefetcher).isDefined) {
    nDCachePorts += 1
  }
}

trait CanHaveL1PrefetcherModule extends HasHellaCacheModule { this:
RocketTileModuleImp =>
  val outer: CanHaveL1Prefetcher with HasTileParameters
  implicit val p: Parameters

  val prefetchOpt = p(BuildL1Prefetcher).map(_(p))
  val prefetchPort = prefetchOpt.map { pfu =>
    val dmem = Wire(new HellaCacheIO()(p))
    pfu.io.dmem.nack := dmem.s2_nack
    pfu.io.dmem.req.ready := dmem.req.ready
    dmem.req.valid := pfu.io.dmem.req.valid
    dmem.req.bits.addr := pfu.io.dmem.req.bits.addr
    dmem.req.bits.idx.foreach(_ := dmem.req.bits.addr)
    dmem.req.bits.tag := DontCare
```

```
    dmem.req.bits.cmd := Mux(pfu.io.dmem.req.bits.write, M_PFW, M_PFR)
    dmem.req.bits.size := log2Ceil(xLen/8).U
    dmem.req.bits.signed := false.B
    dmem.req.bits.phys := false.B
    dmem.req.bits.no_alloc := false.B
    dmem.req.bits.no_xcpt := false.B
    dmem.req.bits.data := DontCare
    dmem.req.bits.dv := DontCare
    dmem.req.bits.mask := 0.U
    dmem.s1_data.data := DontCare
    dmem.s1_data.mask := 0.U
    // Tie off unused control signals
    dmem.s1_kill := false.B
    dmem.s2_kill := false.B
    dmem.keep_clock_enabled := pfu.io.dmem.req.valid
    pfu.io.dmem.resp.valid := dmem.resp.valid // Memory Value Resp
    pfu.io.dmem.resp.bits  := dmem.resp.bits
    dmem
  }
}

// See MemoryOpConstants for encoding of `cmd` field
class CoreMemReq(implicit p: Parameters) extends CoreBundle()(p) with
HasCoreMemOp

class L1PrefetchReq(implicit p: Parameters) extends CoreBundle()(p) {
  val addr = UInt(coreMaxAddrBits.W) // Must be aligned to XLEN
  val write = Bool()
}

class L1PrefetcherIO(implicit p: Parameters) extends CoreBundle()(p) {
  val cpu = new Bundle {
    val req = Flipped(Valid(new CoreMemReq))
    val miss = Input(Bool()) // Core request from 2 cycles ago has
missed
  }
  val dmem = new Bundle {
    val req = Decoupled(new L1PrefetchReq)
    val nack = Input(Bool()) // Prefetch request from 2 cycles ago is
rejected
    val resp = Flipped(Valid(new HellaCacheResp))
  }
}
```

```scala
abstract class L1Prefetcher(implicit p: Parameters) extends
CoreModule()(p) {
  val io = IO(new L1PrefetcherIO)
}

/**
 * Naive implementation of the one-block prefetch-on-miss scheme
 */
class ExampleL1Prefetcher(implicit p: Parameters) extends L1Prefetcher
{

  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)
  val s1_addr = s1_req.addr(coreMaxAddrBits-1, lgCacheBlockBytes)
  val s1_addr_next = s1_addr + 16.U // Compute address of next block

  val s2_req = RegNext(s1_req)
  s2_req.addr := Cat(s1_addr_next, 0.U(lgCacheBlockBytes.W))

  // Keep prefetch request active after a miss is signaled if L1D is not
  // immediately ready to accept it
  val prefetch_count = RegInit(0.U(64.W))
  val miss_hold = RegInit(false.B)
  when (io.cpu.miss) {
    miss_hold := true.B
  }
  // Clear flag after a prefetch request goes through or if another
  // memory request is arriving the next cycle
  when (s1_valid || io.dmem.req.ready) {
    miss_hold := false.B
    prefetch_count := prefetch_count + 1.U
  }

  printf(p"prefetch_count = ${prefetch_count}\n")
  io.dmem.req.valid := io.cpu.miss || miss_hold
  io.dmem.req.bits.addr := s2_req.addr
  io.dmem.req.bits.write := isWrite(s2_req.cmd)


  dontTouch(io.dmem.nack)
}


/**
 * TODO: Implement your custom prefetcher
```

```
 */
class CustomL1Prefetcher(implicit p: Parameters) extends L1Prefetcher {
  // See L1PrefetcherIO bundle for IO port definitions
  io.dmem.req.valid := false.B // FIXME
}

class NextLineL1Prefetcher(implicit p: Parameters) extends L1Prefetcher
{
  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)
  val s1_addr = s1_req.addr(coreMaxAddrBits-1, lgCacheBlockBytes)
  val s1_addr_next = s1_addr + 1.U // Compute address of next block

  val s2_valid = RegNext(s1_valid, false.B)
  val s2_req = RegNext(s1_req)
  s2_req.addr := Cat(s1_addr_next, 0.U(lgCacheBlockBytes.W))
  val curr_req_addr = RegInit(0.U(coreMaxAddrBits.W))

  val access_hold = RegInit(false.B)
  when (s2_valid) {
    access_hold := true.B
  }
  when (io.dmem.req.ready) {
    access_hold := false.B
  }

  io.dmem.req.valid := io.cpu.req.valid || access_hold
  io.dmem.req.bits.addr := s2_req.addr
  io.dmem.req.bits.write := isWrite(s2_req.cmd)
  curr_req_addr := s2_req.addr


  dontTouch(io.dmem.nack)
}

class DuplexStrideL1Prefetcher(val threshold: UInt)(implicit p:
Parameters) extends L1Prefetcher {
  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)

  val s2_valid = RegNext(s1_valid, false.B)
  val s2_req = RegNext(s1_req)

  val access_hold = RegInit(false.B)
```

```
  val last_read_addr = RegInit(0.U(coreMaxAddrBits.W))
  val last_write_addr = RegInit(0.U(coreMaxAddrBits.W))
  val read_confidence = RegInit(0.U(32.W))
  val write_confidence = RegInit(0.U(32.W))
  val read_stride = RegInit(0.U(coreMaxAddrBits.W))
  val write_stride = RegInit(0.U(coreMaxAddrBits.W))
  val prefetch_block_addr = RegInit(0.U(coreMaxAddrBits.W))
  val prefetch_is_write = RegInit(false.B)
  val prefetch = WireInit(false.B)

  when (s2_valid) {
    when (s2_req.cmd === M_XRD) {
      val diff = s2_req.addr - last_read_addr
      when (read_confidence === 0.U) {
        read_confidence := 1.U
        read_stride := diff
      } .otherwise {
        when (diff === read_stride) { // Issue Prefetch
          val new_conf = Mux(read_confidence === threshold, threshold,
read_confidence + 1.U)
          read_confidence := new_conf

          prefetch := new_conf === threshold
          val prefetch_addr = (s2_req.addr + 3.U * read_stride).asUInt
          prefetch_block_addr := Cat(prefetch_addr(coreMaxAddrBits-1,
lgCacheBlockBytes), 0.U(lgCacheBlockBytes.W))
          prefetch_is_write := false.B
        } .otherwise {
          read_confidence := 1.U
          read_stride := diff
        }
      }
      last_read_addr := s2_req.addr
    }
    .otherwise {
      val diff = s2_req.addr - last_write_addr
      when (write_confidence === 0.U) {
        write_confidence := 1.U
        write_stride := diff
      } .otherwise {
        when (diff === write_stride) { // Issue Prefetch
          val new_conf = Mux(write_confidence === threshold, threshold,
write_confidence + 1.U)
          write_confidence := new_conf
```

```
          prefetch := new_conf === threshold
          val prefetch_addr = (s2_req.addr + 3.U * write_stride).asUInt
          prefetch_block_addr := Cat(prefetch_addr(coreMaxAddrBits-1,
lgCacheBlockBytes), 0.U(lgCacheBlockBytes.W))
          prefetch_is_write := true.B
        } .otherwise {
          write_confidence := 1.U
          write_stride := diff
        }
      }
      last_write_addr := s2_req.addr
    }
  }

  val prefetch_count = RegInit(0.U(64.W))
  when (prefetch && s2_valid) {
    access_hold := true.B
  }
  when (io.dmem.req.fire() || io.dmem.nack || io.cpu.req.valid) {
    access_hold := false.B
    when(io.dmem.req.fire()) {
      prefetch_count := prefetch_count + 1.U
    }
  }

  printf(p"prefetch_count = ${prefetch_count}\n")
  io.dmem.req.valid := access_hold
  io.dmem.req.bits.addr := prefetch_block_addr
  io.dmem.req.bits.write := prefetch_is_write


  dontTouch(io.dmem.nack)
}

class DuplexMultiStrideL1Prefetcher(val threshold:UInt, val
maxConfidence: UInt)(implicit p: Parameters) extends L1Prefetcher {
  // Stage 1: Confidence Calc
  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)

  val last_read_addr = RegInit(0.U(coreMaxAddrBits.W))
  val last_write_addr = RegInit(0.U(coreMaxAddrBits.W))
  val read_confidence = RegInit(0.U(32.W))
```

```
  val write_confidence = RegInit(0.U(32.W))
  val read_stride = RegInit(0.U(coreMaxAddrBits.W))
  val write_stride = RegInit(0.U(coreMaxAddrBits.W))

  val s1_task_start = RegInit(0.U.asTypeOf(new CoreMemReq))
  val s1_task_num = RegInit(0.U(32.W))
  val s1_task_stride = RegInit(0.U(coreMaxAddrBits.W))
  val s1_new_task = WireInit(false.B)

  when (s1_valid) {
    when (s1_req.cmd === M_XRD) {
      val diff = s1_req.addr - last_read_addr
      when (read_confidence === 0.U) {
        read_confidence := 1.U
        read_stride := diff
      } .otherwise {
        when (diff === read_stride) {
          val new_conf = Mux(read_confidence === maxConfidence,
read_confidence, read_confidence + 1.U)
          read_confidence := new_conf

          when (new_conf > threshold) {
            s1_task_start := s1_req
            s1_task_num := new_conf - threshold
            s1_task_stride := read_stride
            s1_new_task := true.B
          }
        } .otherwise {
          read_confidence := 1.U
          read_stride := diff

          s1_task_num := 0.U
        }
      }
      last_read_addr := s1_req.addr
    }
    .otherwise {
      val diff = s1_req.addr - last_write_addr
      when (write_confidence === 0.U) {
        write_confidence := 1.U
        write_stride := diff
      } .otherwise {
        when (diff === write_stride) {
```

```
          val new_conf = Mux(write_confidence === maxConfidence,
write_confidence, write_confidence + 1.U)
          write_confidence := new_conf

          when (new_conf > threshold) {
            s1_task_start := s1_req
            s1_task_num := new_conf - threshold
            s1_task_stride := write_stride
            s1_new_task := true.B
          }
        } .otherwise {
          write_confidence := 1.U
          write_stride := diff

          s1_task_num := 0.U
        }
      }
      last_write_addr := s1_req.addr
    }
  }

  // Stage 2: Prefetch Issue
  val access_hold = RegInit(false.B)
  val s2_task_start = RegNext(s1_task_start)
  val s2_task_num = RegNext(s1_task_num)
  val s2_task_stride = RegNext(s1_task_stride)
  val s2_new_task = RegNext(s1_new_task)
  val s2_task_done = RegInit(0.U(32.W))

  when (s2_task_num > s2_task_done) {
    access_hold := true.B
  }
  when (io.dmem.req.fire() || io.cpu.req.valid || io.dmem.nack) {
    access_hold := false.B
    when (io.dmem.req.fire()) {
    }
  }

  val prefetch_addr = s2_task_start.addr + (s2_task_done + 1.U) *
s2_task_stride
  io.dmem.req.valid := access_hold
  io.dmem.req.bits.addr := Cat(prefetch_addr(coreMaxAddrBits-1,
lgCacheBlockBytes), 0.U(lgCacheBlockBytes.W))
  io.dmem.req.bits.write := isWrite(s2_task_start.cmd)
```

```
  when (io.dmem.req.fire()) {
    when (s2_task_done < s2_task_num) {
      s2_task_done := s2_task_done + 1.U
    }
  }


  dontTouch(io.dmem.nack)
}

class ReadMultiStrideL1Prefetcher(val threshold:UInt, val
maxConfidence: UInt)(implicit p: Parameters) extends L1Prefetcher {
  // Stage 1: Confidence Calc
  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)

  val last_read_addr = RegInit(0.U(coreMaxAddrBits.W))
  val read_confidence = RegInit(0.U(32.W))
  val read_stride = RegInit(0.U(coreMaxAddrBits.W))

  val s1_task_start = RegInit(0.U.asTypeOf(new CoreMemReq))
  val s1_task_num = RegInit(0.U(32.W))
  val s1_task_stride = RegInit(0.U(coreMaxAddrBits.W))
  val s1_new_task = WireInit(false.B)

  when (s1_valid) {
    when (s1_req.cmd === M_XRD) {
      val diff = s1_req.addr - last_read_addr
      when (read_confidence === 0.U) {
        read_confidence := 1.U
        read_stride := diff
      } .otherwise {
        when (diff === read_stride) {
          val new_conf = Mux(read_confidence === maxConfidence,
read_confidence, read_confidence + 1.U)
          read_confidence := new_conf

          when (new_conf > threshold) {
            s1_task_start := s1_req
            s1_task_num := new_conf - threshold
            s1_task_stride := read_stride
            s1_new_task := true.B
          }
        } .otherwise {
```

```
          read_confidence := 1.U
          read_stride := diff

          s1_task_num := 0.U
        }
      }
      last_read_addr := s1_req.addr
    }
  }

  // Stage 2: Prefetch Issue
  val access_hold = RegInit(false.B)
  val s2_task_start = RegNext(s1_task_start)
  val s2_task_num = RegNext(s1_task_num)
  val s2_task_stride = RegNext(s1_task_stride)
  val s2_new_task = RegNext(s1_new_task)
  val s2_task_done = RegInit(0.U(32.W))

  when (s2_task_num > s2_task_done) {
    access_hold := true.B
  }
  when (io.dmem.req.fire() || io.cpu.req.valid || io.dmem.nack) {
    access_hold := false.B
    when (io.dmem.req.fire()) {
    }
  }

  val prefetch_addr = s2_task_start.addr + (s2_task_done + 1.U) *
s2_task_stride
  io.dmem.req.valid := access_hold
  io.dmem.req.bits.addr := Cat(prefetch_addr(coreMaxAddrBits-1,
lgCacheBlockBytes), 0.U(lgCacheBlockBytes.W))
  io.dmem.req.bits.write := false.B
  when (io.dmem.req.fire()) {
    when (s2_task_done < s2_task_num) {
      s2_task_done := s2_task_done + 1.U
    }
  }


  dontTouch(io.dmem.nack)
}
```

```
class WriteMultiStrideL1Prefetcher(val threshold:UInt, val
maxConfidence: UInt)(implicit p: Parameters) extends L1Prefetcher {
  // Stage 1: Confidence Calc
  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)

  val last_write_addr = RegInit(0.U(coreMaxAddrBits.W))
  val write_confidence = RegInit(0.U(32.W))
  val write_stride = RegInit(0.U(coreMaxAddrBits.W))

  val s1_task_start = RegInit(0.U.asTypeOf(new CoreMemReq))
  val s1_task_num = RegInit(0.U(32.W))
  val s1_task_stride = RegInit(0.U(coreMaxAddrBits.W))
  val s1_new_task = WireInit(false.B)

  when (s1_valid) {
    when (s1_req.cmd === M_XWR) {
      val diff = s1_req.addr - last_write_addr
      when (write_confidence === 0.U) {
        write_confidence := 1.U
        write_stride := diff
      } .otherwise {
        when (diff === write_stride) {
          val new_conf = Mux(write_confidence === maxConfidence,
write_confidence, write_confidence + 1.U)
          write_confidence := new_conf

          when (new_conf > threshold) {
            s1_task_start := s1_req
            s1_task_num := new_conf - threshold
            s1_task_stride := write_stride
            s1_new_task := true.B
          }
        } .otherwise {
          write_confidence := 1.U
          write_stride := diff

          s1_task_num := 0.U
        }
      }
      last_write_addr := s1_req.addr
    }
  }
```

```
  // Stage 2: Prefetch Issue
  val access_hold = RegInit(false.B)
  val s2_task_start = RegNext(s1_task_start)
  val s2_task_num = RegNext(s1_task_num)
  val s2_task_stride = RegNext(s1_task_stride)
  val s2_new_task = RegNext(s1_new_task)
  val s2_task_done = RegInit(0.U(32.W))

  when (s2_task_num > s2_task_done) {
    access_hold := true.B
  }
  when (io.dmem.req.fire() || io.cpu.req.valid || io.dmem.nack) {
    access_hold := false.B
    when (io.dmem.req.fire()) {
    }
  }

  val prefetch_addr = s2_task_start.addr + (s2_task_done + 1.U) *
s2_task_stride
  io.dmem.req.valid := access_hold
  io.dmem.req.bits.addr := Cat(prefetch_addr(coreMaxAddrBits-1,
lgCacheBlockBytes), 0.U(lgCacheBlockBytes.W))
  io.dmem.req.bits.write := false.B
  when (io.dmem.req.fire()) {
    when (s2_task_done < s2_task_num) {
      s2_task_done := s2_task_done + 1.U
    }
  }


  dontTouch(io.dmem.nack)
}

class DMPL1Prefetcher(implicit p: Parameters) extends L1Prefetcher {
  // Stage 1: Preprocess
  // Memory Region Detection
  val s1_valid = RegNext(io.cpu.req.valid, false.B)
  val s1_req = RegEnable(io.cpu.req.bits, io.cpu.req.valid)
  val s1_mem_addr_cap = RegInit(0x7F.U(coreMaxAddrBits.W))
  val s1_mem_addr_floor = RegInit(~0.U(coreMaxAddrBits.W))
  when (s1_valid) {
    when (s1_req.addr > s1_mem_addr_cap) {
      s1_mem_addr_cap := s1_req.addr
    }
```

```
    when (s1_req.addr < s1_mem_addr_floor) {
      s1_mem_addr_floor := s1_req.addr
    }
  }
  // Memory Response Retrieval
  val s1_mem_valid = RegNext(io.dmem.resp.valid, false.B)
  val s1_mem_data = RegEnable(io.dmem.resp.bits.data,
io.dmem.resp.valid)
  val s1_data_is_addr = RegInit(false.B)
  when (s1_mem_valid) {
    val in_range = (s1_mem_data <= s1_mem_addr_cap) && (s1_mem_data >=
s1_mem_addr_floor)
    val is_aligned = s1_mem_data(lgCacheBlockBytes-1, 0) === 0.U
    s1_data_is_addr := in_range && is_aligned
  }

  // Stage 2: Prefetch Issue
  val s2_prefetch = RegNext(s1_data_is_addr, false.B)
  val s2_addr = RegNext(s1_mem_data, 0.U)
  val access_hold = RegInit(false.B)
  when (s2_prefetch && !io.dmem.req.valid) {
    access_hold := true.B
  }
  when (io.dmem.req.fire() || io.cpu.req.valid || io.dmem.nack) {
    access_hold := false.B
  }
  io.dmem.req.valid := access_hold
  io.dmem.req.bits.addr := Cat(s2_addr(coreMaxAddrBits-1,
lgCacheBlockBytes), 0.U(lgCacheBlockBytes.W))
  io.dmem.req.bits.write := false.B

  dontTouch(io.dmem.nack)
}

class MultiL1PrefetcherWrapper(implicit p: Parameters) extends
L1Prefetcher {
  val nextLine     = Module(new NextLineL1Prefetcher)
  val readStride   = Module(new ReadMultiStrideL1Prefetcher(threshold =
3.U, maxConfidence = 8.U))
  val writeStride  = Module(new WriteMultiStrideL1Prefetcher(threshold =
3.U, maxConfidence = 8.U))
  val dmp          = Module(new DMPL1Prefetcher)

  val subPrefetchers = Seq(readStride, writeStride, dmp, nextLine)
```

```
  // Broadcast CPU signals to all sub-prefetchers
  for (pfu <- subPrefetchers) {
    pfu.io.cpu.req   := io.cpu.req
    pfu.io.cpu.miss  := io.cpu.miss
  }

  // Arbitrate dmem requests using a round-robin arbiter
  val arb = Module(new Arbiter(new L1PrefetchReq,
subPrefetchers.length))
  for ((pfu, i) <- subPrefetchers.zipWithIndex) {
    arb.io.in(i) <> pfu.io.dmem.req
  }
  io.dmem.req <> arb.io.out

  // Count and print total prefetches issued
  val prefetch_count = RegInit(0.U(64.W))
  when (io.dmem.req.fire()) {
    prefetch_count := prefetch_count + 1.U
  }
  printf(p"prefetch_count = ${prefetch_count}\n")
  printf(p"winner = ${arb.io.chosen}\n")

  val s1_chosen = RegNext(arb.io.chosen)
  val s2_chosen = RegNext(s1_chosen)

  for ((pfu, i) <- subPrefetchers.zipWithIndex) {
    pfu.io.dmem.nack      := io.dmem.nack && (s2_chosen === i.U)
    pfu.io.dmem.resp.valid := io.dmem.resp.valid
    pfu.io.dmem.resp.bits  := io.dmem.resp.bits
  }
}

/**
 * Wrapper around C++ software model
 */
class ModelL1Prefetcher(implicit p: Parameters) extends L1Prefetcher {
  val model = Module(new ModelL1PrefetcherHarness(
    addrBits = io.cpu.req.bits.addr.getWidth,
    tagBits = io.cpu.req.bits.tag.getWidth,
    cmdBits = io.cpu.req.bits.cmd.getWidth,
    sizeBits = io.cpu.req.bits.size.getWidth))

  model.io.clock := clock
```

```
  model.io.reset := reset
  model.io.cpu := io.cpu
  io.dmem <> model.io.dmem
}

/**
 * Blackbox for Verilog DPI harness
 */
class ModelL1PrefetcherHarness(addrBits: Int, tagBits: Int, cmdBits:
Int, sizeBits: Int)(implicit p: Parameters)
    extends BlackBox(Map(
      "ADDR_BITS" -> IntParam(addrBits),
      "TAG_BITS" -> IntParam(tagBits),
      "CMD_BITS" -> IntParam(cmdBits),
      "SIZE_BITS" -> IntParam(sizeBits),
      "LINE_SHIFT" -> IntParam(log2Up(p(CacheBlockBytes)))))
    with HasBlackBoxResource {

  val io = IO(new L1PrefetcherIO {
    val clock = Input(Clock())
    val reset = Input(Bool())
  })

  addResource("/vsrc/L1Prefetcher.v")
  addResource("/csrc/L1Prefetcher.cc")
}
```